Complete E-Commerce Platform – Advanced Documentation

Developer: Davix HDM

Contact: +254768784909

Email : davismcintyre5@gmail.com

GitHub: https://github.com/Davismcintyre5

Live URLs

- Customer Frontend: https://electrostore.pxxl.click
- Admin Dashboard: https://electrostore-admin.pxxl.click
- Backend API: https://electrostoreserver.pxxl.click

Table of Contents

1. [Project Overview] (#project-overview)
2. [Features](#features)
   - 2.1 [Customer Features] (#customer-features)
   - 2.2 [Admin Features] (#admin-features)
   - 2.3 [Backend Features] (#backend-features)
3. [System Architecture] (#system-architecture)
   - 3.1 [Component Diagram] (#component-diagram)
   - 3.2 [Data Flow] (#data-flow)
4. [Technology Stack] (#technology-stack)
5. [Getting Started (Local Development)] (#getting-started-local-development)
   - 5.1 [Prerequisites](#prerequisites)
   - 5.2 [Installation](#installation)
6. [Configuration](#configuration)
   - 6.1 [Backend Environment Variables] (#backend-environment-variables)

- 6.2 [Frontend Environment Variables] (#frontend-environment-variables)
- 6.3 [CORS Configuration] (#cors-configuration)
7. [API Documentation] (#api-documentation)
- 7.1 [Authentication Endpoints] (#authentication-endpoints)
- 7.2 [Customer Endpoints] (#customer-endpoints)
- 7.3 [Admin Endpoints] (#admin-endpoints)
- 7.4 [M-Pesa Endpoints] (#m-pesa-endpoints)
8. [Database Schema] (#database-schema)
- 8.1 [User Model] (#user-model)
- 8.2 [Product Model] (#product-model)
- 8.3 [Order Model] (#order-model)
- 8.4 [Other Models] (#other-models)
9. [Real-time Notifications (Socket.IO)] (#real-time-notifications-socketio)
- 9.1 [Events Emitted] (#events-emitted)
- 9.2 [Client Integration] (#client-integration)
10. [M-Pesa Integration] (#m-pesa-integration)
- 10.1 [Payment Flow] (#payment-flow)
- 10.2 [Callback Handling] (#callback-handling)
11. [Admin Dashboard Deep Dive] (#admin-dashboard-deep-dive)
- 11.1 [Role-Based Access Control] (#role-based-access-control)
- 11.2 [Temporary Password Flow] (#temporary-password-flow)
- 11.3 [Reports & Printing] (#reports--printing)
12. [Customer Frontend Deep Dive] (#customer-frontend-deep-dive)
- 12.1 [Key User Flows] (#key-user-flows)
- 12.2 [Address Management] (#address-management)
- 12.3 [Cart & Checkout] (#cart--checkout)
13. [Security Considerations] (#security-considerations)
- 13.1 [Authentication & Authorization] (#authentication--authorization)
- 13.2 [Data Validation] (#data-validation)
- 13.3 [CORS](#cors)

- 13.4 [Rate Limiting] (#rate-limiting)
- 13.5 [Helmet.js] (#helmetjs)
14. [Performance Optimizations] (#performance-optimizations)
- 14.1 [Database Indexing] (#database-indexing)
- 14.2 [Pagination](#pagination)
- 14.3 [Image Optimization] (#image-optimization)
- 14.4 [Caching](#caching)
15. [Testing](#testing)
- 15.1 [Backend Testing] (#backend-testing)
- 15.2 [Frontend Testing] (#frontend-testing)
16. [Deployment Guide] (#deployment-guide)
- 16.1 [Backend Deployment (Render)] (#backend-deployment-render)
- 16.2 [Frontend Deployment (Netlify)] (#frontend-deployment-netlify)
- 16.3 [Environment Variables in Production] (#environment-variables-in-production)
- 16.4 [Custom Domains & SSL] (#custom-domains--sol)
17. [Maintenance & Monitoring] (#maintenance--monitoring)
- 17.1 [Logging](#logging)
- 17.2 [Monitoring](#monitoring)
- 17.3 [Backups](#backups)
- 17.4 [Updates & Patches] (#updates--patches)
18. [Troubleshooting Common Issues] (#troubleshooting-common-issues)
- 18.1 [Database Connection Errors] (#database-connection-errors)
- 18.2 [CORS Errors] (#cors-errors)
- 18.3 [M-Pesa Callback Failures] (#m-pesa-callback-failures)
19. [Developer Information] (#developer-information)
20. [License](#license)

---

1. Project Overview

Electro Store is a modern, full-stack e-commerce platform designed for selling electronics. It provides a seamless shopping experience for customers and a powerful management interface for administrators. The system is built with scalability, security, and ease of use in mind. It supports real-time notifications, M-Pesa payments, and a responsive design that works on all devices.

The project is split into three independent applications:

- Backend API – Handles business logic, authentication, database operations, and real-time events.
- Admin Dashboard – A React-based single-page application for store administrators to manage products, orders, customers, and staff.
- Customer Frontend – A React-based storefront where users can browse products, manage their cart, checkout, and track orders.

All three communicate via HTTP and WebSocket's, with the backend acting as the central hub.

---

2. Features

2.1 Customer Features

- User Registration & Login – Customers can create an account and log in securely. Passwords are hashed using bcrypt.
- Product Browsing – View products with filtering by category, search by name/description, and pagination.
- Product Details – Detailed view with multiple images, price, description, and stock status.
- Shopping Cart – Add products, adjust quantities, remove items. Cart is persisted in the database.
- Wishlist – Save favorites products for later.
- Checkout – Select a delivery address (up to 3 addresses per user, with one default), choose payment method (M-Pesa or simulated card), and place an order.
- M-Pesa Payment – Integrated STK push for mobile payments. Users receive a prompt on their phone.
- Order History – View all past orders with status and details.

- Printable Invoices – Each order can be printed as a formatted invoice.

- Real-time Notifications – Receive updates when order status changes (e.g., shipped, delivered).

- Account Management – Update profile information and change password.

- Address Book – Add, edit, delete addresses; set a default address.

- Responsive Design – Works on desktop, tablet, and mobile.

2.2 Admin Features

- Secure Login – Role-based access for admin, manager, and cashier.

- Dashboard – Key metrics: orders today, revenue today, pending orders, low stock items, total customers.

- Product Management – Create, read, update, delete products. Upload multiple images per product.

- Promo Code Management – Create and manage promotional codes with percentage or fixed discounts, validity periods, usage limits.

- Order Management – View all orders, filter by status, search by order ID. Update order status (pending → confirmed → processing → shipped → delivered). Print invoices.

- Transaction Listing – View all M-Pesa transactions, search by receipt number or order ID.

- Customer Management – View list of registered customers, search by name/email.

- Staff User Management – Admins can create, edit, delete staff users (admin, manager, cashier). New staff receive a temporary password and must change it on first login.

- System Settings – Configure store name, contact information, currency, VAT rate.

- Notifications Panel – View system notifications and mark as read.

- Reports – Generate sales reports (daily/monthly/yearly) and top-selling products. Print reports.

- Search & Pagination – All list pages support searching and pagination for large datasets.

- Responsive Layout – Collapsible sidebar for mobile, adaptive tables.

2.3 Backend Features

- RESTful API – Clean, resource-oriented endpoints.

- JWT Authentication – Stateless authentication with token expiration.

- Role-Based Middleware – Protect routes based on user roles (customer, admin, manager, cashier).

- File Upload – Multer handles product image uploads, stored in `public/uploads`.

- M-Pesa Integration – STK push initiation and callback handling with transaction status updates.

- Real-time Notifications – Socket.IO emits events for new orders and order status changes.

- Input Validation – Request validation using Joi (or custom middleware).

- Error Handling – Centralized error handler with appropriate HTTP status codes.

- Logging – Winston logs errors and info to files.

- Database Models – Mongoose schemas with indexes for performance.

- Utility Scripts – `seed.js`, `createAdmin.js`, `checkAdmins.js`, `dropDB.js` for development and maintenance.

- CORS – Configurable allowed origins.

---

3. System Architecture

3.1 Component Diagram

```
┌──────────────────────┐     ┌──────────────────────┐
┌──────────────────────┐
│                      │     │                      │
│ Customer             │─────▶│ Backend API          │◀─────│ Admin                │
│ Frontend             │     │ (Node.js)            │     │ Dashboard            │
│ (React)              │     │                      │     │ (React)              │
│                      │     │                      │     │                      │
└──────────────────────┘     └──────────────────────┘
└──────────────────────┘
                                        │
                                        ▼
                                ┌──────────────────────┐
                                │ MongoDB              │
                                │ (Atlas)              │
```

└──────────────────────────┘

```

### 3.2 Data Flow

1. Customer places an order:
   - Frontend sends order data to `POST /api/customer/checkout`.
   - Backend validates cart and address, creates order, clears cart.
   - If payment method is M-Pesa, backend initiates STK push via Safaricom API.
   - Backend emits `newOrder` event to all connected admins.
   - Upon payment success (callback), backend updates order status and emits `paymentSuccess` to the customer.

2. Admin updates order status:
   - Admin sends `PATCH /api/admin/orders/:id/status` with new status.
   - Backend updates order and emits `orderStatusUpdated` to the specific customer.
   - Customer receives real-time update and sees new status.

---

## 4. Technology Stack

| Area | Technology | Purpose |
|---|---|---|
| Backend Runtime | Node.js | JavaScript runtime |
| Backend Framework | Express | Web framework |
| Database | MongoDB (Mongoose ODM) | Data persistence |
| Authentication | JWT (JSON Web Tokens) | Stateless auth |
| File Upload | Multer | Handle multipart/form-data |
| Real-time | Socket.IO | Bi-directional event communication |
```

Logging	Winston	Application logging
HTTP Client	Axios	External API calls (M-Pesa)
Frontend (both)	React	UI library
Build Tool	Vite	Fast development and production builds
Styling	Tailwind CSS	Utility-first CSS
Icons	Heroicons	SVG icons
HTTP Client (front)	Axios	API requests
Routing (front)	React Router v6	Client-side routing
State Management	React Context API	Global state (auth, cart)

---

5. Getting Started (Local Development)

5.1 Prerequisites

- Node.js v18+ and npm
- Git
- MongoDB (local installation or Atlas account)
- A code editor (VS Code recommended)
- (Optional) Postman or similar for API testing

5.2 Installation

Backend

```bash
git clone https://github.com/Davismcintyre5/electrostoreserver.git
cd electrostorer/server   # or backend folder
npm install
```

```
cp .env.example .env
# Edit .env with your values (see Configuration section)
npm run dev
```

The backend will start on `http://localhost:5000`.

Admin Frontend

```bash
git clone https://github.com/Davismcintyre5/electrostore-admin.git
cd electrostore-admin
npm install
cp .env.example .env
# Set VITE_API_URL=http://localhost:5000/api
npm run dev
```

Admin runs on `http://localhost:3001`.

Customer Frontend

```bash
git clone https://github.com/Davismcintyre5/electrostore-customer.git
cd electrostore-customer
npm install
cp .env.example .env
# Set VITE_API_URL=http://localhost:5000/api
npm run dev
```

Customer runs on `http://localhost:3000`.

Make sure the backend is running before launching the frontends.

---

6. Configuration

6.1 Backend Environment Variables

Create a `.env` file in the `backend` folder. Below is a comprehensive example:

```env
# Server
NODE_ENV=development
PORT=5000

# Database
MONGODB_URI=mongodb+srv://username:password@cluster.mongodb.net/Ecommerce?retryWrites=true&w=majority

# JWT
JWT_SECRET=your_very_strong_secret_key_at_least_32_chars
JWT_EXPIRE=7d

# M-Pesa (Sandbox)
MPESA_CONSUMER_KEY=your_consumer_key
MPESA_CONSUMER_SECRET=your_consumer_secret
MPESA_PASSKEY=your_passkey
```

MPESA_SHORTCODE=174379

MPESA_CALLBACK_URL=https://yourdomain.com/api/mpesa/callback # Must be HTTPS in production

CORS (Frontend URLs)

CLIENT_URL=http://localhost:3000

ADMIN_URL=http://localhost:3001

```

### 6.2 Frontend Environment Variables

Admin `.env`

```env

VITE_API_URL=http://localhost:5000/api

```

Customer `.env`

```env

VITE_API_URL=http://localhost:5000/api

```

In production, replace `localhost` with your actual backend domain.

6.3 CORS Configuration

The backend uses the `CLIENT_URL` and `ADMIN_URL` environment variables to allow cross-origin requests. These are set in `server.js`:

```javascript

const allowedOrigins = [

  process.env.CLIENT_URL || 'http://localhost:3000',

```
  process.env.ADMIN_URL || 'http://localhost:3001',
];
app.use(cors({ origin: allowedOrigins, credentials: true }));
```

---

7. API Documentation

Full API reference is available in the [backend README](./backend/README.md). Below are the key endpoints with examples.

7.1 Authentication Endpoints

`POST /api/auth/register`

Register a new customer.

Request Body:

```json
{
  "name": "John Doe",
  "email": "john@example.com",
  "password": "secret123",
  "phone": "254700000000"
}
```

Response (201):

```json
{
```

```
  "token": "eyJhbGciOiJIUzI1NiIs...",
  "user": {
    "id": "60d...",
    "name": "John Doe",
    "email": "john@example.com",
    "role": "customer"
  }
}
```

`POST /api/auth/login`

Login for all users.

Request Body:

```json
{
  "email": "john@example.com",
  "password": "secret123"
}
```

Response (200):

```json
{
  "token": "eyJhbGciOiJIUzI1NiIs...",
  "requiresPasswordChange": false,
  "user": { ... }
}
```

If `requiresPasswordChange` is true, the frontend must redirect to a password change page.

`PATCH /api/auth/change-password`

Change password for authenticated user.

Headers: `Authorization: Bearer <token>`

Request Body:

```json
{
  "currentPassword": "oldpass",
  "newPassword": "newpass"
}
```

Response (200): `{ "message": "Password changed successfully" }`

7.2 Customer Endpoints

`GET /api/customer/products`

Get paginated list of products. Supports search, category filter, and pagination.

Query Parameters: `?search=phone&category=Phones&page=1&limit=20`

Response:

```json
{
  "products": [...],
  "total": 45,
  "page": 1,
```

```
  "pages": 3
}
```

`GET /api/customer/cart`

Get current user's cart.

Response: Cart object with populated items.

`POST /api/customer/cart`

Add item to cart.

Request Body: `{ "productId": "...", "quantity": 1 }`

`DELETE /api/customer/cart/:productId`

Remove item from cart.

`POST /api/customer/checkout`

Place an order (requires address and payment method).

Request Body:

```json
{
  "addressId": "...",
  "paymentMethod": "mpesa"
}
```

Response: The created order object.

`GET /api/customer/orders`

Get all orders for the logged-in user.

`GET /api/customer/orders/:id`

Get a single order with populated items and address.

7.3 Admin Endpoints

All admin endpoints are prefixed with `/api/admin` and require an admin or manager role.

`GET /api/admin/products`

Get all products (including inactive). Supports search and pagination.

`POST /api/admin/products`

Create a new product (multipart/form-data with images).

`PUT /api/admin/products/:id`

Update product (supports image uploads).

`DELETE /api/admin/products/:id`

Delete product (removes associated images).

`GET /api/admin/orders`

Get all orders with filtering by status and search.

`PATCH /api/admin/orders/:id/status`

Update order status. Emits socket event.

Request Body: `{ "status": "shipped" }`

`GET /api/admin/users`

List staff users (admin, manager, cashier).

`POST /api/admin/users`

Create a new staff user (admin only). Automatically generates temporary password.

`GET /api/admin/settings`

Get system settings.

`PUT /api/admin/settings`

Update settings (admin only).

`GET /api/admin/reports/sales`

Get sales report aggregated by day/month/year.

7.4 M-Pesa Endpoints

`POST /api/mpesa/stkpush`

Initiate STK push. Requires authentication (customer).

Request Body:

```json
{
  "phone": "254700000000",
  "amount": 1000,
  "orderId": "..."
}
```

Response: Transaction object with pending status.

`POST /api/mpesa/callback`

Callback endpoint for Safaricom (public, no auth). Updates transaction and order status.

---

8. Database Schema

MongoDB collections with Mongoose schemas.

8.1 User Model

```javascript
{
  name: String, required
  email: String, unique, lowercase
  password: String, required
  role: { type: String, enum: ['customer','admin','manager','cashier'], default: 'customer' }
  mustChangePassword: Boolean, default: false
  phone: String
  isActive: Boolean, default: true
  createdAt: Date
}
```

8.2 Product Model

```javascript
{
  name: String, required
```

```
  description: String
  price: Number, required
  category: String
  stock: Number, default: 0
  images: [String]  // array of image paths/URLs
  isActive: Boolean, default: true
  createdAt: Date
}
```

8.3 Order Model

```javascript
{
  user: { type: ObjectId, ref: 'User' }
  items: [{
    product: { type: ObjectId, ref: 'Product' },
    quantity: Number,
    price: Number
  }]
  totalAmount: Number
  shippingAddress: { type: ObjectId, ref: 'Address' }
  paymentMethod: String
  paymentStatus: { type: String, enum: ['pending','paid','failed'] }
  orderStatus: { type: String, enum: ['pending','confirmed','processing','shipped','delivered','cancelled'] }
  transaction: { type: ObjectId, ref: 'Transaction' }
  createdAt: Date
}
```

8.4 Other Models

- Address – linked to user, with fields like `label`, `recipientName`, `addressLine1`, `city`, `isDefault`, etc. Enforces max 3 per user.
- Cart – one per user, items array.
- Wishlist – one per user, array of product IDs.
- Promo – code, discount type, value, validity, usage limit.
- Transaction – order reference, amount, M-Pesa receipt, status.
- Notification – user, type, title, message, read status.
- SystemSettings – singleton document with store configuration.

---

9. Real-time Notifications (Socket.IO)

Socket.IO is used to push real-time updates to connected clients.

9.1 Events Emitted

Event	Emitted To	When	Payload Example
`newOrder`	Admins room	A customer places an order	`{ orderId: "..." }`
`orderStatusUpdated`	Specific customer	Admin updates order status	`{ orderId: "...", status: "shipped" }`
`paymentSuccess`	Specific customer	M-Pesa callback confirms payment	`{ orderId: "..." }`

9.2 Client Integration

Frontend connection:

```javascript
```

```
import io from 'socket.io-client';

const socket = io(import.meta.env.VITE_API_URL.replace('/api', ''), {
  auth: { token: localStorage.getItem('token') }
});

socket.on('orderStatusUpdated', (data) => {
  // Update UI, show notification
});
```

---

10. M-Pesa Integration

10.1 Payment Flow

1. Customer selects M-Pesa during checkout and provides phone number.
2. Frontend calls `POST /api/mpesa/stkpush` with phone, amount, and order ID.
3. Backend generates access token, constructs STK push request, and sends to Safaricom API.
4. Safaricom responds with `CheckoutRequestID`; backend stores transaction as pending.
5. Customer receives STK push on their phone, enters PIN.
6. Safaricom calls the callback URL with the result.

10.2 Callback Handling

- Callback endpoint `/api/mpesa/callback` receives the result.
- Backend updates transaction status to `completed` or `failed`.
- If successful, order payment status is updated to `paid`.
- A socket event `paymentSuccess` is emitted to the customer.

- The customer frontend can redirect to order confirmation.

Callback security: The endpoint is public (no auth) but relies on the secrecy of the callback URL. In production, it should be HTTPS.

---

11. Admin Dashboard Deep Dive

11.1 Role-Based Access Control

- Admin: Full access – can manage products, orders, users, settings, reports.
- Manager: Can manage products, orders, view reports, but cannot create/delete staff.
- Cashier: Can view orders and update status, but cannot edit products or view financial reports.

The frontend checks the user's role from the JWT and conditionally renders menu items. Backend routes are protected by the `roleCheck` middleware.

11.2 Temporary Password Flow

When an admin creates a new staff user:

- Backend generates a random temporary password (e.g., 8 characters).
- User is created with `mustChangePassword: true`.
- On first login, the response includes `requiresPasswordChange: true`.
- Frontend forces a password change before allowing access to the dashboard.

11.3 Reports & Printing

- Sales report: aggregated by day, month, or year, with optional date range.
- Top products: list of best-selling items.

- Each report page has a Print button that opens a printer-friendly version (hidden navigation, clean tables) using `window.print()`.

---

12. Customer Frontend Deep Dive

12.1 Key User Flows

- Browse → Add to Cart → Checkout – Smooth transitions with cart persisted.
- Product search – Debounced input to avoid excessive API calls.
- Wishlist – Heart icon toggles product in wishlist.

12.2 Address Management

- Users can add up to 3 addresses.
- The first address added becomes default automatically.
- Default can be changed via "Set as default" button.
- During checkout, only addresses are shown; default is pre-selected.

12.3 Cart & Checkout

- Cart uses React Context for global state; updates trigger re-renders.
- Checkout page validates address selection and payment method before placing order.
- After order placement, cart is cleared and user is redirected to order details.

---

13. Security Considerations

13.1 Authentication & Authorization

- Passwords hashed with bcrypt (salt rounds = 10).
- JWT tokens signed with a strong secret; stored in `localStorage` (consider httpOnly cookies for higher security).
- Middleware `auth.js` verifies token and attaches user to `req`.
- `roleCheck.js` ensures only users with required roles access certain endpoints.

13.2 Data Validation

- Incoming data is validated using Joi schemas (or custom validation) to prevent injection attacks.
- Mongoose schemas enforce data types and required fields.

13.3 CORS

- Only allowed origins (set via environment variables) can access the API.
- Credentials (cookies) are allowed only if needed.

13.4 Rate Limiting

(Optional but recommended) – Implement `express-rate-limit` to prevent brute-force attacks on login and registration.

13.5 Helmet.js

Consider adding `helmet` middleware to set secure HTTP headers.

---

14. Performance Optimizations

14.1 Database Indexing

Indexes are created on frequently queried fields:

- `User.email` (unique)
- `Product.name`, `Product.category`
- `Order.user`, `Order.createdAt`
- `Transaction.order`, `Transaction.mpesaReceiptNumber`

14.2 Pagination

All list endpoints support `page` and `limit` parameters to limit response size.

14.3 Image Optimization

- Images are resized and compressed before upload (not implemented yet – could add sharp.js).
- Use responsive images (`srcset`) in frontend.

14.4 Caching

- Static assets (images) are served with `Cache-Control` headers.
- API responses can be cached with Redis for frequently accessed data (future enhancement).

---

15. Testing

15.1 Backend Testing

- Unit tests for controllers and models using Jest.
- Integration tests for API endpoints.

- Mock M-Pesa calls to avoid external dependencies.

15.2 Frontend Testing

- Component tests with React Testing Library.
- End-to-end tests with Cypress for critical flows (login, add to cart, checkout).

Currently, tests are not fully implemented – this is a future improvement.

---

16. Deployment Guide

16.1 Backend Deployment (Render)

1. Push code to a GitHub repository.
2. On Render, create a new Web Service.
3. Connect the repository.
4. Set build command: `npm install`
5. Start command: `npm start`
6. Add environment variables from your `.env` (including `NODE_ENV=production`).
7. Deploy.

Ensure your MongoDB Atlas IP whitelist includes Render's IPs (or set to allow all for simplicity).

16.2 Frontend Deployment (Netlify)

1. Build each frontend:
   ```bash
   cd admin
   ```

```
npm run build   # creates dist folder
```

2. Drag and drop the `dist` folder to Netlify, or connect via Git.

3. Set environment variable `VITE_API_URL` to your live backend URL (e.g., `https://electrostoreserver.pxxl.click/api`).

Repeat for the customer frontend.

16.3 Environment Variables in Production

- Backend: Set all variables in the hosting dashboard.

- Frontend: For Vite apps, environment variables must be prefixed with `VITE_` and are embedded at build time. So you need to rebuild if the API URL changes.

16.4 Custom Domains & SSL

- Point your domain (e.g., `electrostore.pxxl.click`) to Netlify nameservers.

- Netlify automatically provisions SSL certificates.

- For Render, add a custom domain in the dashboard; SSL is auto-provided.

---

17. Maintenance & Monitoring

17.1 Logging

- Winston logs errors and info to `logs/error.log` and `logs/combined.log`.

- In production, consider using a logging service like Logtail or Papertrail.

17.2 Monitoring

- Use uptime monitoring (e.g., UptimeRobot) for the backend and frontends.
- Monitor server metrics (CPU, memory) via hosting provider.

17.3 Backups

- MongoDB Atlas provides automated backups. Enable them.
- Regularly export critical collections if needed.

17.4 Updates & Patches

- Keep dependencies updated (`npm outdated`, `npm update`).
- Monitor for security vulnerabilities with `npm audit`.

---

18. Troubleshooting Common Issues

18.1 Database Connection Errors

- Error: `MongoNetworkError`
- Solution: Check MongoDB URI, network whitelist, and ensure database is running.

18.2 CORS Errors

- Error: `No 'Access-Control-Allow-Origin' header`
- Solution: Verify `CLIENT_URL` and `ADMIN_URL` are set correctly in backend `.env` and match the frontend origin.

18.3 M-Pesa Callback Failures

- Issue: Callback not received or transaction not updated.
- Check: Callback URL must be publicly accessible (not localhost). Use a service like ngrok for testing.
- Ensure the callback endpoint does not require authentication.

---

19. Developer Information

Name: Davix HDM

Phone: +254768784909

Email: davismcintyre5@gmail.com

GitHub: https://github.com/Davismcintyre5

For technical support, feature requests, or collaboration, please contact via email or phone.

---

20. License

This project is proprietary and confidential. Unauthorized copying, distribution, or use is strictly prohibited. All rights reserved.

© 2026 ElectroStore.